Persistent Tool Use for Deterministic Execution in Reasoning LMs

Anonymous Authors

Abstract

We present a persistent, sandboxed Python tool for reasoning LMs that enforces deterministic modules, seeded state, and strict call syntax. Failures surface as natural-language traces and permit one structured retry. On math and program-analysis tasks we observe shorter traces and lower latency at comparable accuracy, with a compact, reproducible recipe.

1 Introduction

While tool use helps reliability, common setups are stateless, allow nondeterminism, and leak interpreter echoes that models imitate. We formalize persistent tool use: a long-lived interpreter with constrained calls, deterministic whitelists, per-problem budgets, and retry-once semantics. This yields reproducible traces and variable reuse, reducing recomputation; we detail training signals and study resulting trends. Tool-augmented reasoning spans self-supervised API use, code-as-reasoning, and program-of-thoughts Schick *et al.* [2023]; Gao *et al.* [2023]; Chen *et al.* [2022]. Many pipelines start a fresh process for every call, forcing re-imports, blocking variable reuse, and admitting nondeterminism. We instead keep one sandboxed interpreter per problem with deterministic imports, seed resets, and explicit budgets. Our contributions are a minimal persistent interface with strict syntax and seeds; a compact recipe that rewards successful execution rather than echoing; and an empirical study showing stable accuracy with shorter traces and lower latency.

Preprint.

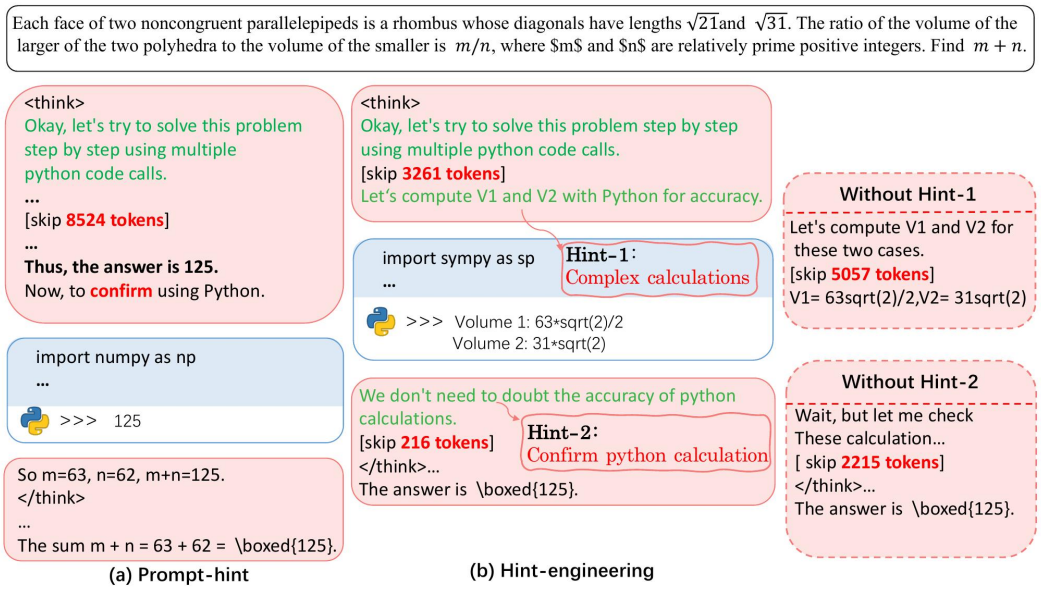

Figure 1: Method overview: long-lived sandbox with deterministic imports, seeded state, and retry-once control.

2 Method

Decoding and control. Tool calls are delimited by reserved tags and decoded with lightweight constraints to ensure syntactic Python. We cap the tool budget per problem and stop on an explicit answer tag. Runtime exceptions are surfaced to the model in natural language and permit a single structured retry; subsequent failures terminate execution and require reasoning without further tool use. The persistent interpreter resets between problems and only whitelists deterministic modules from the standard library.

Safety and determinism. During SFT and RFT we mask interpreter echoes to avoid trivial imitation; in RL, returns depend on final answer correctness and execution success, not echoed outputs. We disable network and time, reset random seeds before each call, and validate stdout length to prevent runaway prints. These decisions simplify reproduction and reduce reward hacking while keeping the post-training recipe compact.

Tool budget and latency. Performance degrades gracefully as the allowed number of tool calls is reduced, with most successful traces using few calls due to earlier invocation and less redundant verification. Persistent execution shortens traces by enabling variable reuse when appropriate, and we observe lower wall-clock latency than non-persistent baselines that repeatedly re-import and recompute. These trends align with the token-efficiency figures and the observed shift toward calculation over verification.

Training recipe. We gather deterministic traces, SFT on tagged calls and terse rationales to encourage early invocation, then brief RFT that rewards correct, executed answers and penalizes gratuitous verification.

3 Experiments

We evaluate on grade-school math (GSM8K) and competition math (MATH), plus small algorithmic coding tasks. Baselines are text-only chain-of-thought, a PAL-style stateless executor, and our persistent tool Gao *et al.* [2023]; Wei *et al.* [2022]; Hendrycks *et al.* [2021]. Metrics: exact match/pass@1, calls per instance, and wall-clock time under fixed budgets in a deterministic environment. For comparability we fix temperature, seeds, and a per-problem call budget. **Protocol.** Shared prompts/decoding; only the executor differs. Matched budgets and reruns verify determinism.

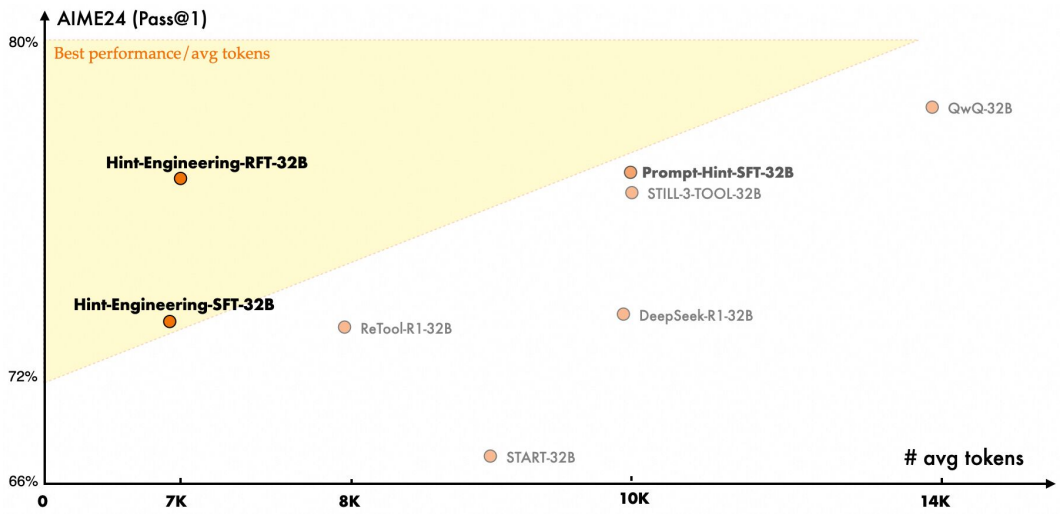

Figure 2: Token efficiency: persistent execution reduces tokens and latency at matched accuracy.

4 Analysis

Ablations indicate which choices matter. Deterministic imports and seeds remove rare nondeterminism; retry-once aids robustness without loops. Error taxonomy: stateless re-imports vs. persistent calculation; failures mostly logic and missing whitelist.

5 Conclusion

Persistent, deterministic tool use yields reproducible traces, stable accuracy, and lower latency. Constraining the API and rewards steers models toward calculation over imitation.

References

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.